Stumpway Testing Day

A manual QA strategy for the stumpway window manager

River + StumpWM-parity stack — Sunday edition

1 Goal and method

Stumpway now has broad feature coverage; this plan puts it through its paces as a **daily driver** under real usage, not just unit conditions. The strategy has three layers:

1. **Regression sweep** — run the automated suite first, so manual testing starts from a known-good base.
2. **Feature circuits** — scripted walks through each feature area with concrete steps and expected results (the bulk of this document).
3. **Soak testing** — use stumpway as your only window manager for the day: browse, edit, chat, take screenshots, switch workspaces under load, and note anything that breaks, flickers, or feels wrong.

Mark each item with a result. Anything that fails: note the section number, what you did, what happened — the capture guide in the appendix turns that into a fixable report.

1.1 Preconditions

☐ Install missing helpers: `sudo pacman -S swaylock swayidle` (lock screen — MOD+Shift+L is a no-op without them). ☐ Fresh login: pick the stumpway (river) session at the SDDM greeter. Watch the first 10 seconds: one bar, one wallpaper process, both autostart apps (variety tray-less in tray, flameshot daemon). ☐ Automated suite green (next section).

1.2 The automated suite (15 min)

Run each from the repo root; every line must end in `0 failed`:

```
for t in workspace-restore workspace-switch-away window-rules \
         hooks-and-eval ipc prefix prompt-and-message floating \
         frames parity quickwins interactivity medium3 bigger1 \
         session-icons bar-guard expose; do
  echo "== $t"; sbcl --non-interactive --load tests/$t.lisp
done
```

Suite	What it covers	Result
frames (57)	Tree ops, dynamic tiling, dumps, refits	☐
expose (22)	Grid, pick paths, exact restore	☐
scroll (27)	Ribbon geometry, clamping, focus recovery	☐
bigger1 (27)	Minor modes, WSE, fuzzy	☐
session-icons (25)	Lock/logout commands, bar icons	☐
the rest (152)	Everything else, per README	☐

Total: 376 checks. If any fail, stop — manual testing is pointless on a broken base.

2 Feature circuits

2.1 B. Dynamic tiling (the new core)

The behavior under the most scrutiny today: every new window should get its own frame.

	Action	Expected
☐	On an empty workspace press MOD+Return twice	First terminal fills the screen; the second splits it side-by-side
☐	MOD+Return again (3rd, 4th, 5th window)	Each arrival splits the frame you are in along its longer axis; the pointer follows the newest window
☐	C-t R (remove split) after several arrivals	Frames merge pairwise into their sibling; windows stack one-per-frame again
☐	Close windows one by one with several open	The empty frame is reclaimed: neighbors grow automatically — no manual C-t R needed
☐	C-t Q (only) with several frames	Whole workspace collapses to one frame; all windows in it, cycle with C-t n
☐	C-t B (balance) after manual resizing	Every split returns to 50/50
☐	Open 8+ windows	Splits stop below twice the minimum frame size (no sliver frames); still stable
☐	C-t n / C-t p inside a multi-window frame	Pulls the next/previous hidden window into the current frame

2.2 C. Frame tree surgery

	Action	Expected
☐	C-t s, C-t S	Manual side-by-side and stacked splits; focus moves to the new empty frame
☐	C-t arrows	Focus moves to the nearest frame in that direction, preferring edge-sharing neighbors
☐	C-t +/-	Dividing line nudges ±30px; iresize (C-t r) then arrows works continuously, Enter commits, Escape cancels
☐	C-t M + arrows	Moves the focused window into that frame
☐	C-t e + arrows	Swaps the current frame's window with the target frame's (focus follows the arrival)
☐	Switch layout: `stumpway-cmd switch-layout master-stack`	Tree rebuilds as master/stack; windows distributed one per frame
☐	`stumpway-cmd dump-frame proj` then open/close windows, then `restore-frame proj`	Layout returns; windows placed by number; missing windows skipped without crash

2.3 D. Expose (C-t Return)

	Action	Expected
☐	With 3+ windows on the workspace: C-t Return	All windows (including hidden-in-frame ones) float into an aspect-fitted grid; a hint message appears
☐	Press 1–9 / 0	That grid slot is picked; layout restores exactly and that window focuses
☐	Arrows, then Return	Focus walks the grid; Return picks the focused window
☐	Click a window in the grid	Pick completes with that window

☐	Escape	Layout and focus restore to pre-expose state exactly — float positions, frame order, focus
☐	Wait 60s mid-expose	Timeout restores safely on its own
☐	One window only: C-t Return	“nothing to expose” message, no state change

2.4 D2. The scroll ribbon (C-t v, niri-style)

	Action	Expected
☐	C-t v with 2-3 windows open	Windows become full-height columns in their tiling order; the bar’s layout module reads “scroll”; a message reports the column count
☐	C-t . / C-t , (period/comma)	Focus moves one column over; the viewport scrolls the minimum distance to keep the focused column fully visible
☐	MOD+. / MOD+,	Viewport scrolls alone in half-screen steps, clamped at both ribbon ends; focus unchanged; the scroll-x message appears
☐	Open a new window while scrolled	It enters as a new column immediately right of the focused one and gets focus
☐	Close the focused window while scrolled	Its column is pruned; focus falls to the vacated slot’s successor, not the first column
☐	Float a column window (C-t f)	It leaves the ribbon (column count drops); the float renders and drags normally
☐	C-t v again	Back to the dynamic tiler; windows re-split into frames as if the ribbon never happened
☐	C-t Return (expose) while scrolled	The overview works over columns; restore returns to the ribbon exactly

2.5 E. Floats and pointer operations

	Action	Expected
☐	C-t f on a tiled window	Floats in place (no jump); “floated” message
☐	MOD+Left-drag a float	Moves; alpha-blended border follows
☐	MOD+Left-drag a **tiled** window	StumpWM-style: it floats first, then moves
☐	MOD+Right-drag a float	Resizes from the bottom-right; respects min/max size hints
☐	Drag a float mostly off-screen, release	Clamped back so 30px stays reachable
☐	Titlebar-drag a CSD app (GIMP toolbox, if installed)	Moves without floating it again
☐	C-t f again to unfloat	Returns to tiling in the current frame
☐	Drag a float across a workspace switch (MOD+1..9 while dragging is fine to skip)	No crash; float lands on its workspace

2.6 F. Workspaces, rules, marks

	Action	Expected
☐	MOD+1..9 / C-t 1..9	Direct switch; bar highlights; hidden windows restore
☐	MOD+Shift+1..9	Moves the focused window and follows it
☐	C-t N / C-t P	Next/previous workspace
☐	`stumpway-cmd renumber` after closing windows	Numbers compact 1..N; C-t ' selects by number
☐	C-t # then `stumpway-cmd move-marked 3`	Marked window (star) moves; marks clear
☐	Set a rule: `(configure '(:window-rules ((:match "foot" :workspace 4))))`, then launch a terminal	It opens on workspace 4; check via C-t w (windowlist shows workspace numbers)
☐	Same with a `:wse` rule: `(:wse (:and (:app-id "foot") (:float nil)) :float t)`	Matching windows float automatically

2.7 G. Prompts, launcher, menus (fuzzy)

	Action	Expected
☐	MOD+D, type “ff”	“Firefox”-style fuzzy hits rank after exact substring matches; the just-installed Variety appears
☐	MOD+W (windowlist)	Numbers shown as `N: [ws] title`; marked windows show a star; Enter jumps to its workspace and focuses
☐	MOD+X (command palette), type “bal”	Fuzzy filters; Enter runs balance; the message window confirms
☐	C-t ' then a number	Selects that window by number
☐	Tab/Up/Down in any prompt	Selection cycles; Escape cancels cleanly every time
☐	Leave a prompt open 30s	Auto-closes; keyboard never trapped

2.8 H. Bar

	Action	Expected
☐	Click workspace numbers in the bar	Switches workspaces via IPC
☐	Click the flameshot icon	Opens flameshot gui
☐	Click the variety icon	Launches/raises Variety preferences
☐	Scroll on the volume module	Volume steps 5%
☐	C-t b / MOD+B	Bar hides and returns; frame trees refit — windows never tiled under or floating over the bar
☐	`(configure '(:bar (:height 30)))`	Bar restarts taller; windows re-tile under it immediately

2.9 I. Session management

	Action	Expected

☐	MOD+Shift+L	Locks (needs swaylock installed); password unlocks
☐	MOD+Shift+E once	“press again to log out” message
☐	MOD+Shift+E twice within 3s	Session ends; SDDM greeter returns; log back in cleanly
☐	`stumpway-cmd suspend`	Machine suspends; wake restores session with input working
☐	MOD+Shift+Q / R	Confirm-style behavior consistent; power actions prompt polkit as configured
☐	C-t q twice / once	Quit needs the second press; a single press just warns

2.10 J. Prefix chains, help, modes

	Action	Expected
☐	Press C-t and pause	Which-key list shows every binding with docs (multi-line message window)
☐	C-t h then descend b/c/f/v/k/w	Each shows its output; k then a key describes it without running it
☐	C-t C (command mode), run 3 commands, Escape	Stays armed between commands; which-key visible; Escape exits
☐	`(enable-minor-mode :test)` with a bound key	Mode binding shadows the prefix binding while enabled; disable restores
☐	C-t m (lastmsg), C-t a (time)	Both show in the message window; lastmsg re-shows the previous text
☐	Unbound key in a chain	“unbound key” message, clean exit
☐	`(set-prefix-key "C-z")`, try C-z then C-t	C-z now opens chains; C-t is inert; set it back with `"(stumpway:set-prefix-key \"C-t\")"`
☐	`(set-user-mod :alt)` and back to `:super`	MOD bindings re-grab in both directions; MOD+drag follows

2.11 K. IPC and REPL robustness

	Action	Expected
☐	`stumpway-cmd current-workspace`, `wse (:float t)`, `renumber`	All reply in the message window without breaking the WM
☐	`stumpway-cmd '(+ 1 2)'`	Evaluates; reply shows in the message window
☐	M-x slime-connect (or `stumpway-cmd`) while windows are moving	No request from a foreign thread errors in the log
☐	Flood: hold MOD+J/K for 10s, then C-t chains immediately	No input loss, no stuck modes
☐	`stumpway-cmd lock` twice in a row	Second lock doesn’t spawn duplicates (swaylock may refuse politely)

3 Soak testing (the afternoon)

Use stumpway as your only session for 3–4 hours. Keep this list open and note anything that matches:

Watch for	Notes
☐ Flicker	Bar redraws, message window flashes, window borders re-drawn repeatedly — check the WM log for manage/render churn (a past bug class)
☐ Focus fights	Auto-focus stealing, clicks not moving focus, expose leaving stale focus
☐ Slow chains	C-t taking > 100ms, launcher scrolling hitching
☐ Workspace drift	Windows landing on wrong workspaces via rules, or losing their frame order after switches
☐ Log growth	Tail `~/.cache/stumpway/stumpway.log` and `bar.log` — idle should add nothing; bursts during drag/resize are expected but should stop on release
☐ Memory	`ps -o rss -p $(pgrep -f stumpway:run)` at start vs end of the afternoon; >2x growth is worth investigating

Suggested load: 8–12 windows across 4+ workspaces, a browser with video, an editor, two terminals, one GIMP-style float if available. Exercise expose, splits, and workspace switches while the video plays.

4 Advanced circuits (if the morning goes well)

- **Stability — protocol stream (the freeze fix):** open and close the launcher, show messages (C-t m), open prompts (MOD+W) repeatedly — 10+ popups. Then check the log: `grep -c "Malformed" ~/.cache/stumpway/stumpway.log` — reads 0 after the v5 compositor cap (any non-zero number means the desync is back; stop and report).
- **Stability — the snapshot storm (the freeze fix):** with the bar running and windows open, run `ls -l --time-style=+%s ~/.cache/stumpway/snapshot.sexp` twice, 10 seconds apart — the mtime must be IDENTICAL while nothing changes (a bumping mtime at idle = the bar feedback storm). Also: `tail -f ~/.cache/stumpway/stumpway.log` should be quiet at idle — a stream of MANAGE-START lines at idle is the storm.
- **Stability — scroll gauntlet:** toggle C-t v, wait 10s (the message expiry), toggle back, wait 10s. The laptop must stay responsive and the log quiet; this exact sequence froze the machine before commit b1e7996.
- **Hot-reload resilience:** `(stumpway:configure '(:bar (:height 26)))` from `stumpway-cmd`, then `ql:quickload` a changed file via the eval prompt (C-t :). The WM should survive; note any stuck mode or lost focus. Recovery lever: `C-t q q` then relogin.
- **Multi-bar safety:** confirm exactly one bar process at all times: `pgrep -fc "stumpway:run-bar"` must read `1` (the fork-bomb guard).
- **Timeout paths:** enter iresize, expose, command mode and just wait 60s in each — all three must restore their state cleanly on expiry.
- **Float + expose interplay:** a pre-existing float plus dynamic tiles, expose, pick, repeat — float geometry must survive every round trip.

5 Appendix A: capture guide (how to report a failure)

- Section number + exact steps + what happened vs expected.
- `tail -50 ~/.cache/stumpway/stumpway.log` (WM) and `~/.cache/stumpway/bar.log`.
- For input/mode bugs: which modes were active (C-t h b lists minor modes), whether a timeout or manual exit was involved.
- One-liner repro if you can: the `stumpway-cmd` or key sequence.

6 Appendix B: quick reference

Key	Action	Key	Action
MOD+Return	Terminal	C-t s / S	Split / stacked split
MOD+J / K	Cycle frame window	C-t R	Remove split
MOD+W / G / X	Windowlist / picker / palette	C-t +/-	Resize split
MOD+D	Launcher (fuzzy)	C-t r	Interactive resize
MOD+B	Toggle bar	C-t Q	Only (one frame)
MOD+1..9	Workspace	C-t B	Balance splits
MOD+Shift+1..9	Move window → ws	C-t Return	Expose
MOD+Shift+L	Lock	C-t f	Float/tile
MOD+Shift+E	Logout (×2)	C-t #	Mark window
C-t q	Quit (×2)	C-t m	Last message
C-t h	Help chains	C-t C	Command mode

Text size: 10pt. Print me, tick me, break things.